



# **Multi-agent Orchestration Patterns in Agent Framework**



**Pasi Huuhka**  
DevOps Architect @ Zure





- We solve business problems with technology
- Designing, building and managing on Azure since 2011
- 120+ experts with an average of 15 years of experience
- Offices in Finland, Belgium, Denmark, UK & Netherlands
- Fully independent, owned by employees



 **Microsoft**  
Solutions Partner  
Digital & App Innovation  
Azure

**Specialist**  
AI Platform on Microsoft Azure  
Build AI Apps on Microsoft  
Azure  
Migrate Enterprise Applications  
to Microsoft Azure

 **Microsoft**  
Solutions Partner  
Data & AI  
Azure

**Specialist**  
Analytics  
Build AI Apps on Microsoft  
Azure  
AI Platform on Microsoft Azure  
Migrate Enterprise Applications  
to Microsoft Azure

 **Microsoft**  
Solutions Partner  
Infrastructure  
Azure

**Specialist**  
Azure Virtual Desktop

 **Microsoft**  
Solutions Partner  
Security

**Specialist**  
Cloud Security  
Threat Protection



**Microsoft®**  
Most Valuable  
Professional

**Microsoft**  
Partner  


2023 Partner of the Year Winner  
Application Innovation  
and Modernization  
Finland

# In this presentation...



Crash course to Agent Framework



Overview of AF orchestration patterns: Sequential, Concurrent, Group Chat, Handoff, Magentic



What each pattern enables and when to use it in real workflows



How to implement these patterns in C#



# Agent Framework

- Microsoft's **unified successor** to both Semantic Kernel & AutoGen
- **Provider-agnostic**. Foundry, Anthropic, Amazon... All plug in identically
- Lets you create **extensible agents** with tools, middleware, and controlled execution
- Designed for **developers**: available in .NET & Python
- Built-in support for memory, embeddings, observability, prompt templates and much more
- **More control, more features** vs tools like Azure AI Foundry Agent Service or Copilot Studio

# Core building blocks

IChatClient (the LLM connection)

- └ ChatClientAgent (an agent wrapping the LLM + instructions + tools)
  - └ Executor<T> (a workflow node — agents ARE executors)
    - └ Workflow (the compiled graph of executors + edges)

# Agents (ChatClientAgent)

```
var scaffolder = AgentFactory.CreateAgent(  
    chatClient,  
    name: "Scaffolder",  
    instructions: """  
        You are a software architect. Based on the user story, suggest a project structure.  
        Use the CreateScaffold tool to generate the structure.  
    """,  
    tools);
```



# Tools (wrap any function)

```
// Create tools from DevWorkflowTools
var tools = new List<AITool>
{
    AIFunctionFactory.Create(DevWorkflowTools.OasGenerate),
    AIFunctionFactory.Create(DevWorkflowTools.RepoCreateBranch),
    AIFunctionFactory.Create(DevWorkflowTools.CreateScaffold),
    AIFunctionFactory.Create(DevWorkflowTools.TestsGenerate),
    AIFunctionFactory.Create(DevWorkflowTools.DocsUpdate)
};
```

```
[Description("Create a feature branch for the implementation")]
1 reference
private static string CreateBranch(string branchName)
{
    return $"Created branch: feature/{branchName}";
}
```



# Workflows

```
// Build sequential workflow
var workflow = AgentWorkflowBuilder.BuildSequential(
    backlogRefiner, scaffolder, apiDesigner, testWriter, docWriter);

// Execute workflow
var messages = new List<ChatMessage> { new(ChatRole.User, prompt) };
var result = await WorkflowRunner.ExecuteAsync(workflow, messages, cli);
```

# Workflows

```
StreamingRun run = await InProcessExecution.RunStreamingAsync(workflow, messages, cancellationToken: cancellationToken);
await run.TrySendMessageAsync(new TurnToken(emitEvents: true));

await foreach (WorkflowEvent evt in run.WatchStreamAsync(cancellationToken))
{
    switch (evt)
    {
        case AgentResponseUpdateEvent e: ...

        case ExecutorCompletedEvent completed: ...

        case RequestInfoEvent requestInfo: ...

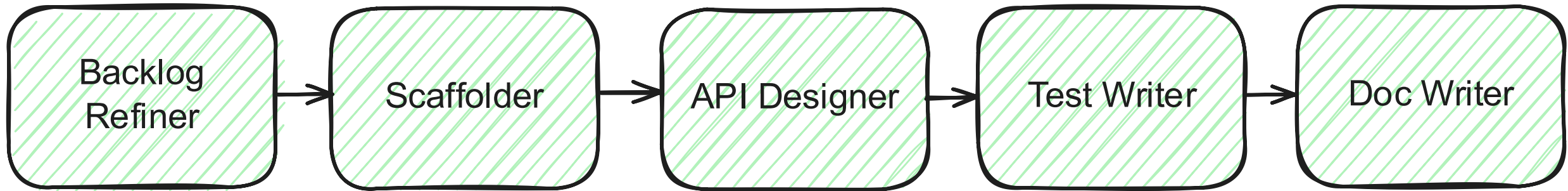
        case WorkflowOutputEvent output: ...

        case ExecutorFailedEvent failed: ...

    }
}
```

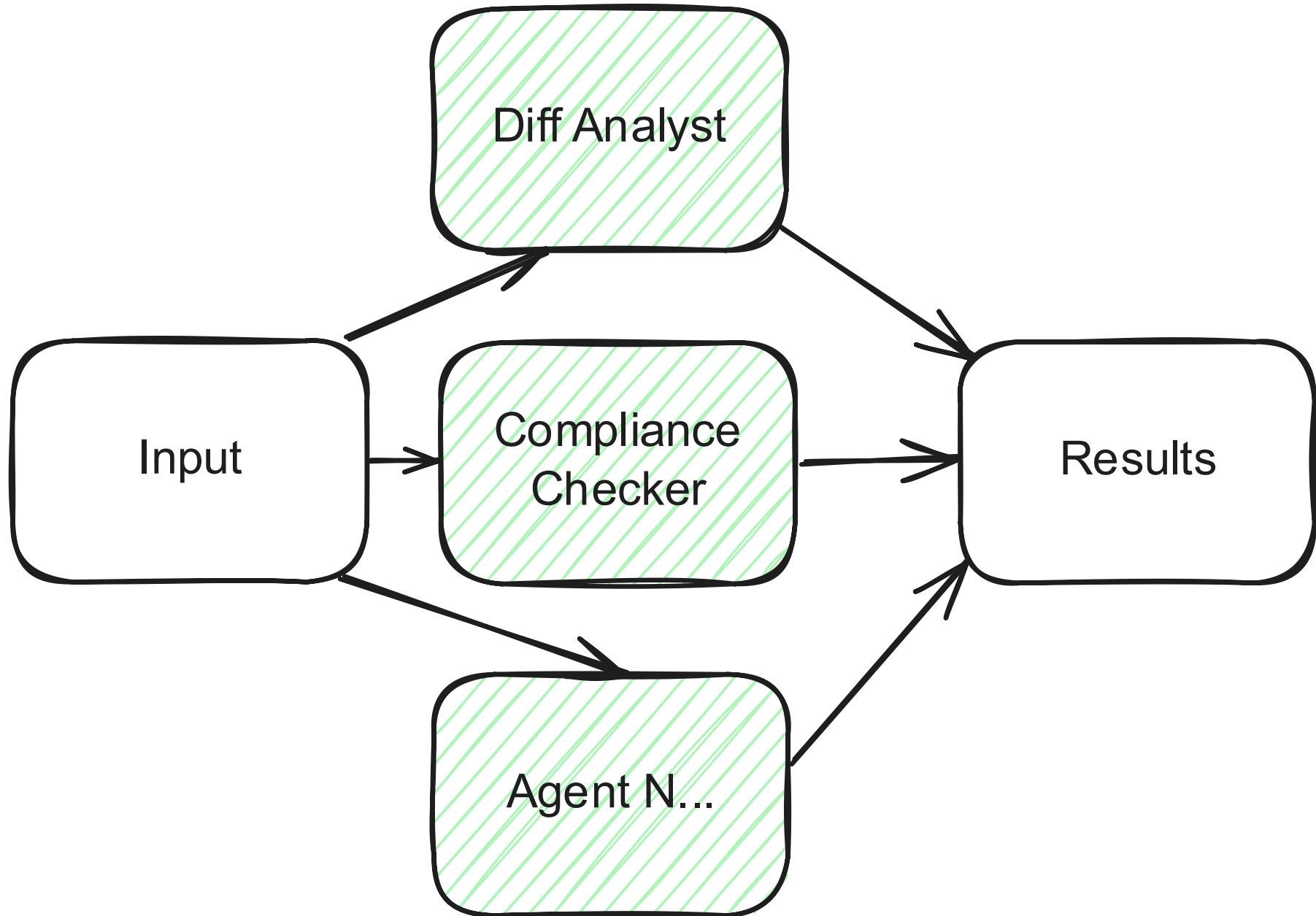
# Orchestration Patterns

# Sequential





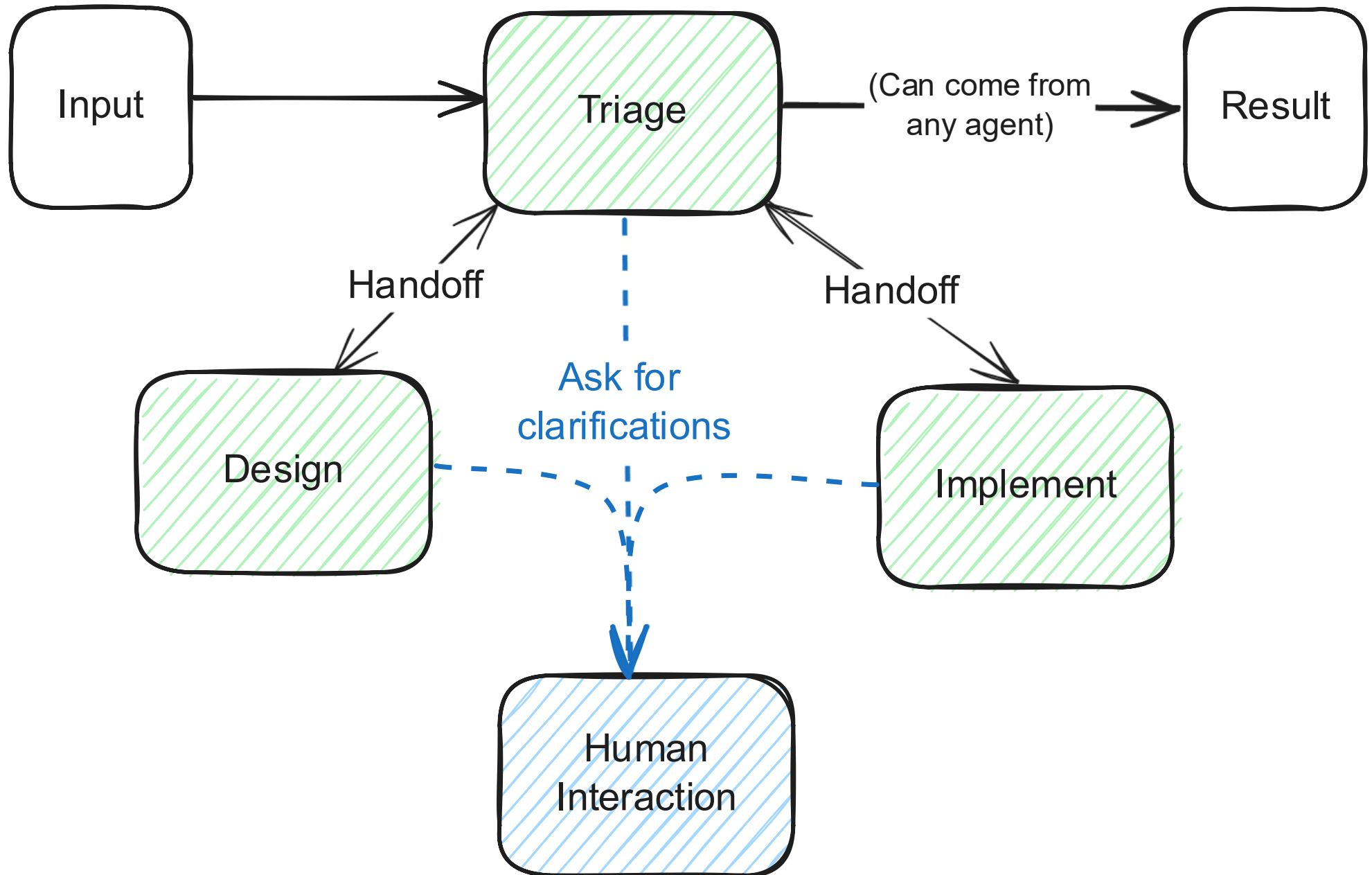
# Concurrent



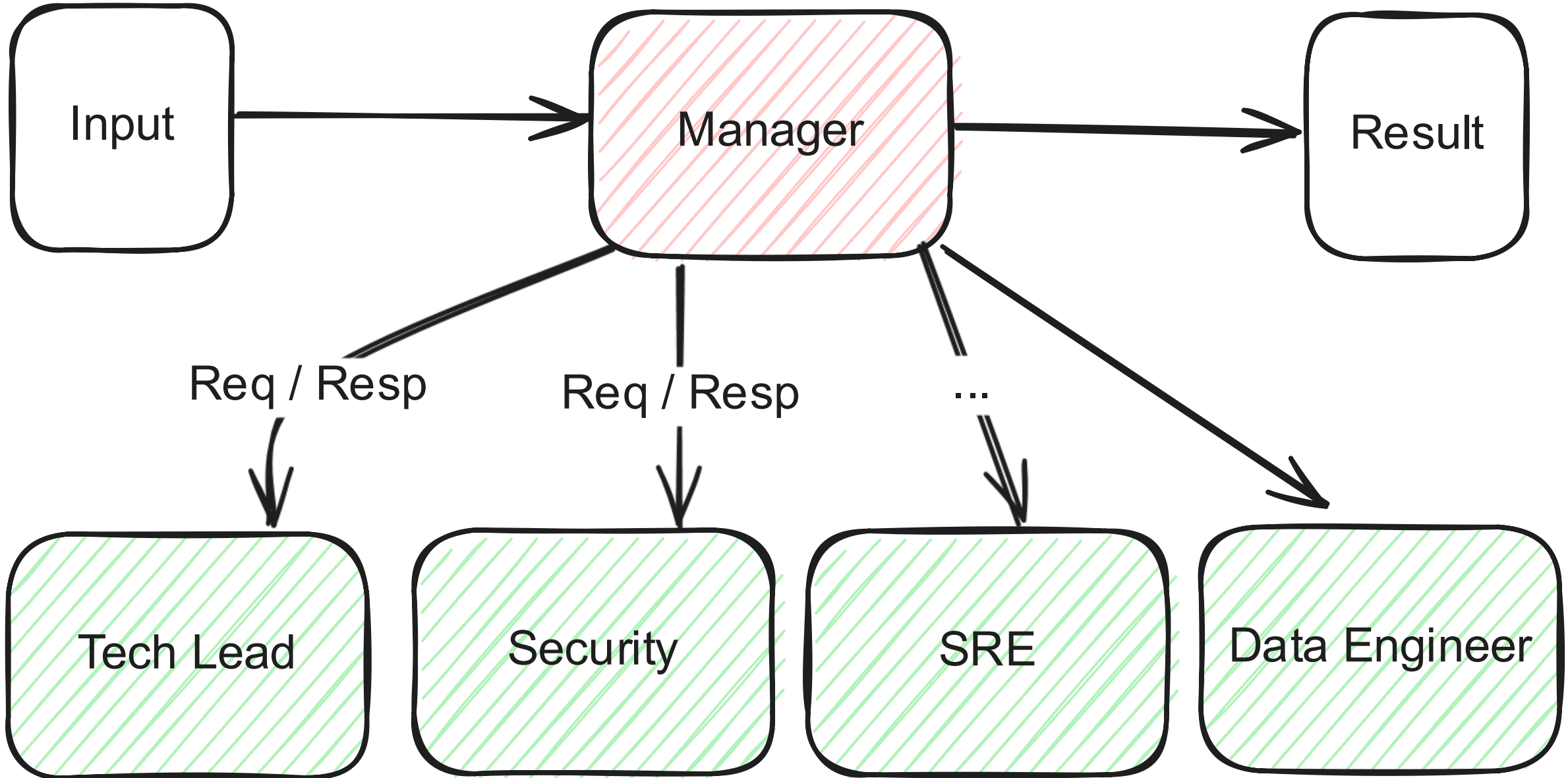
# Graph

??????????

# Handoff

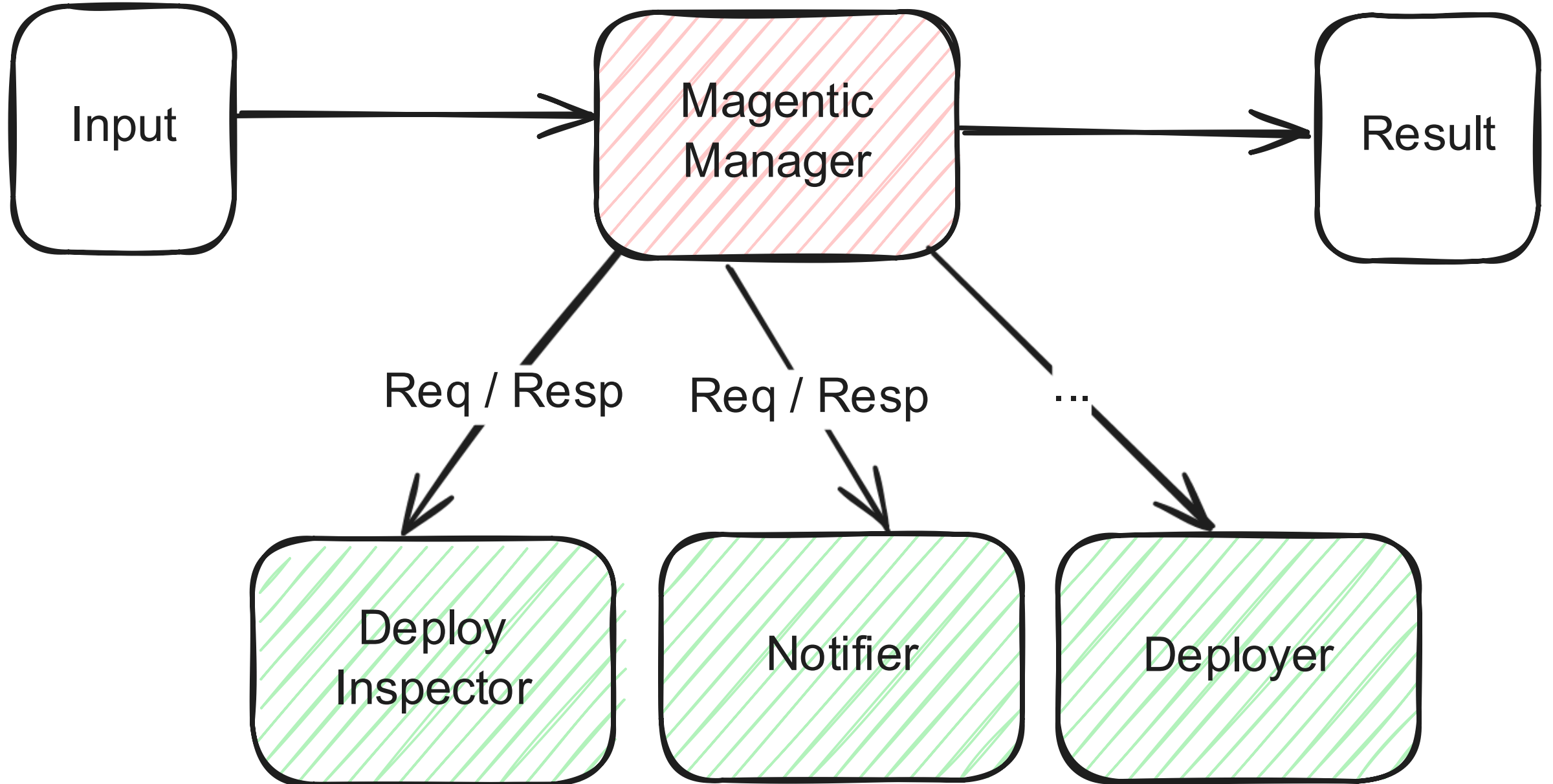


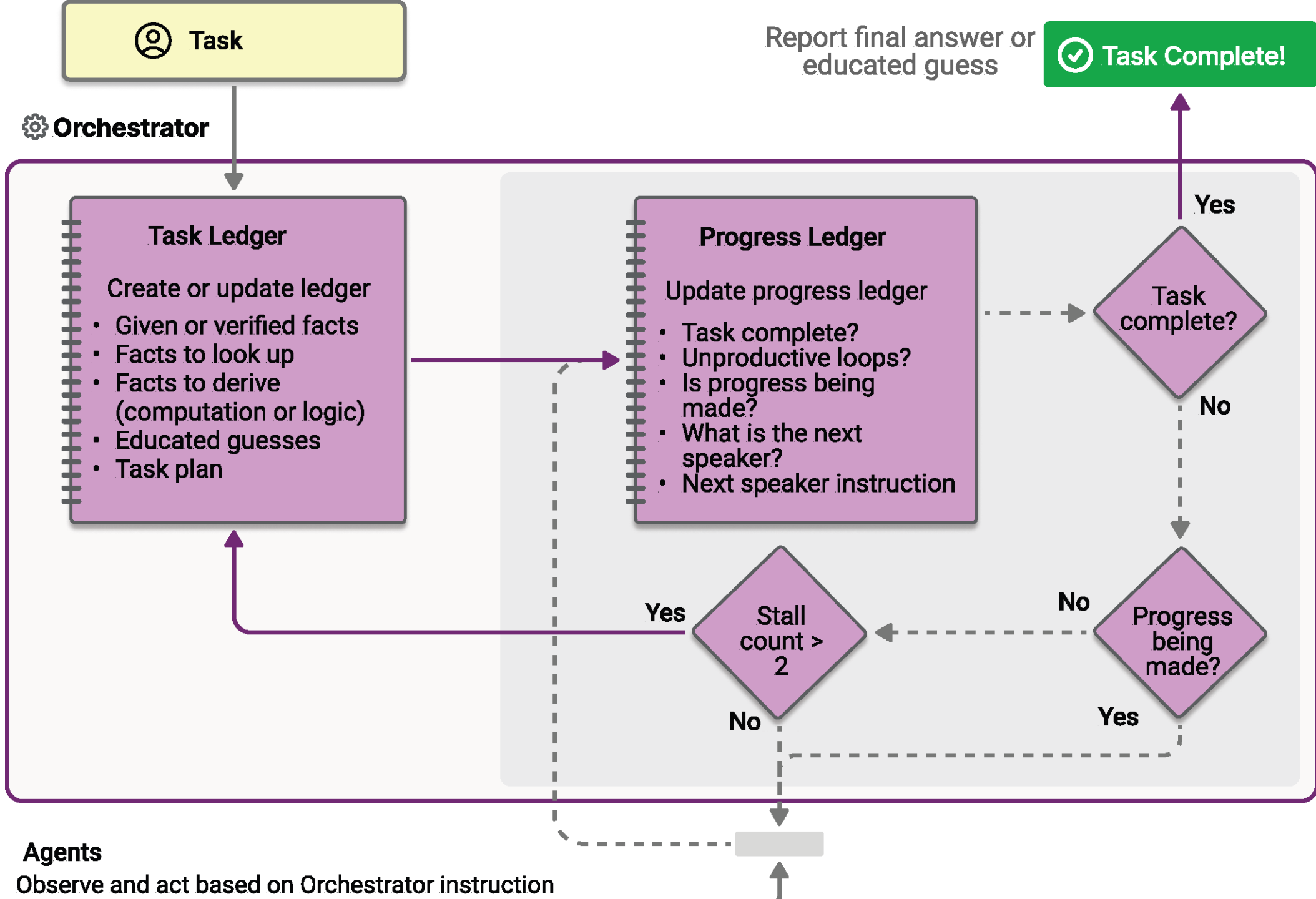
# Group Chat





# Magnetic Orchestration





# Code



***ZURE***